

Kinetic Digital Clock

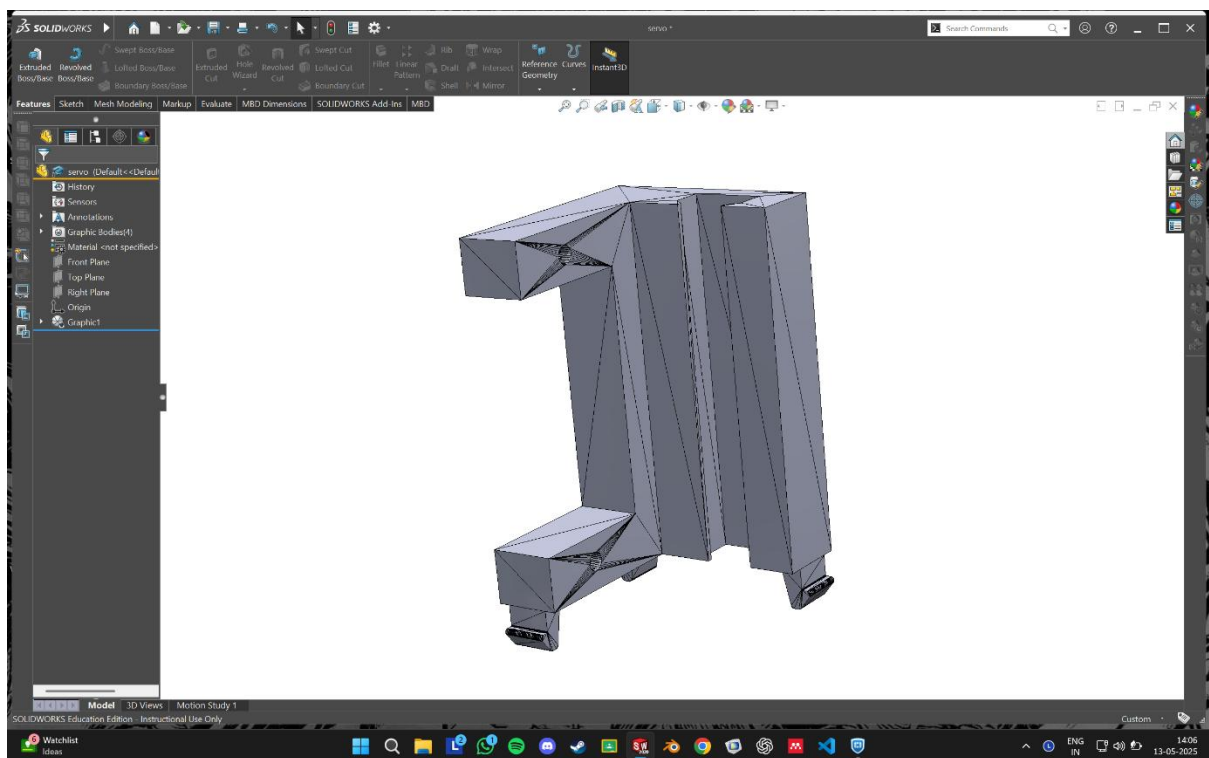
Group Members – Sneha, Rahul, Dhaval, Yash

Process:

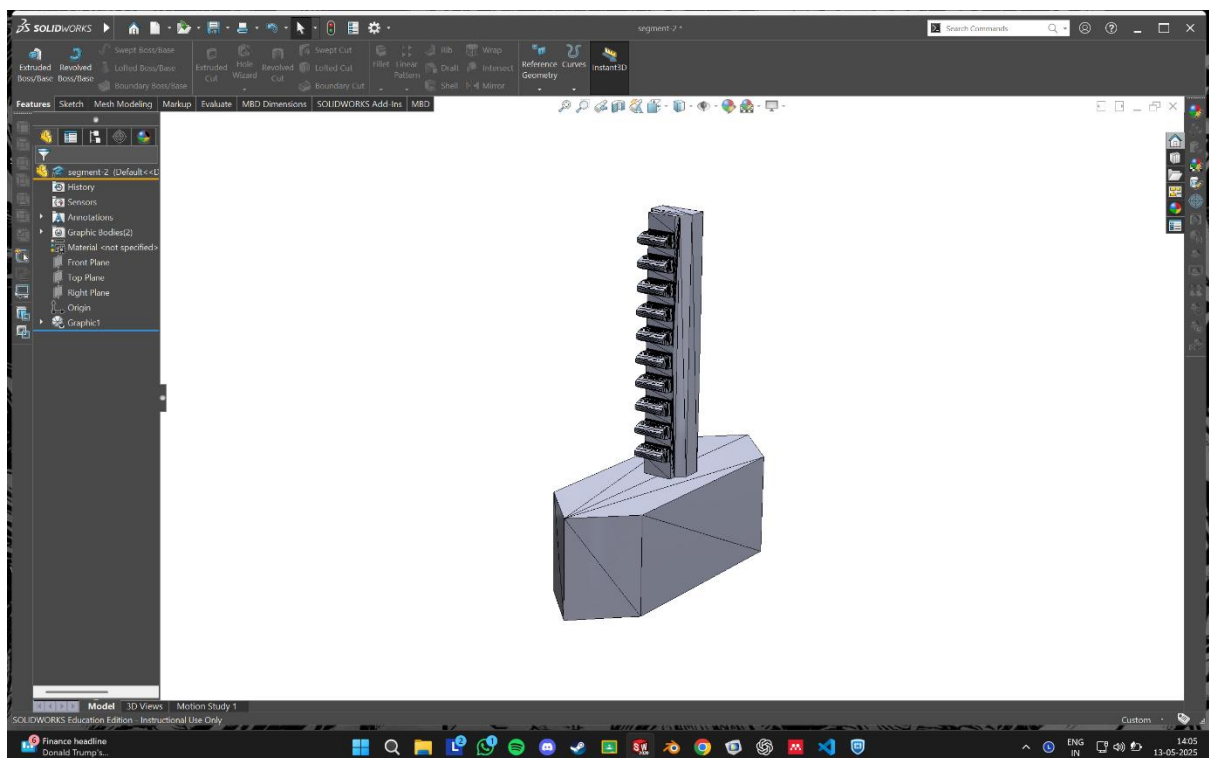
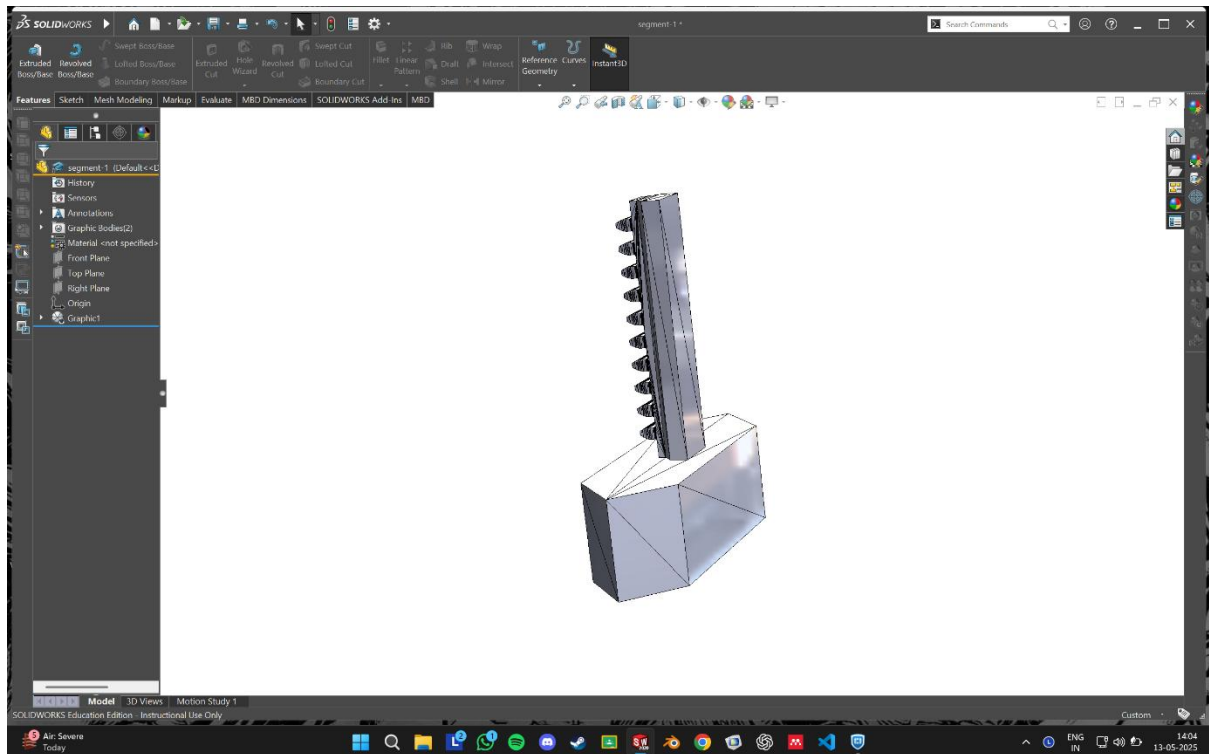
Step 1: For ease of assembly and quick prototyping, the clock is made with modular siling and snap-fit parts. The digits slot into the panel and each digit is made of 7 segments. There are 4 panels in the main body and the back panel slides into the face plate. The segments create a linear motion by moving along the gears. For 3D printing, STL files are made. Orientation of the pieces is essential to have the components look good. Like when the clock faces flat on a steel sheet, it gives them a polished smooth finish that makes it look neat and premium.

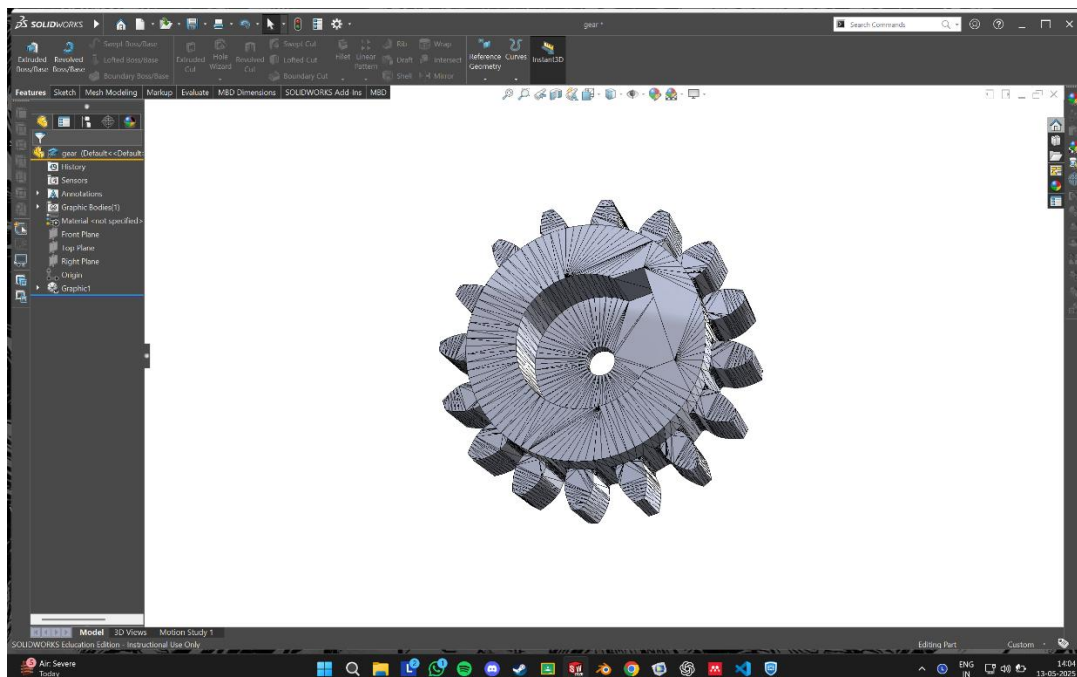
Images of 3D model made:

Servo Bracket:

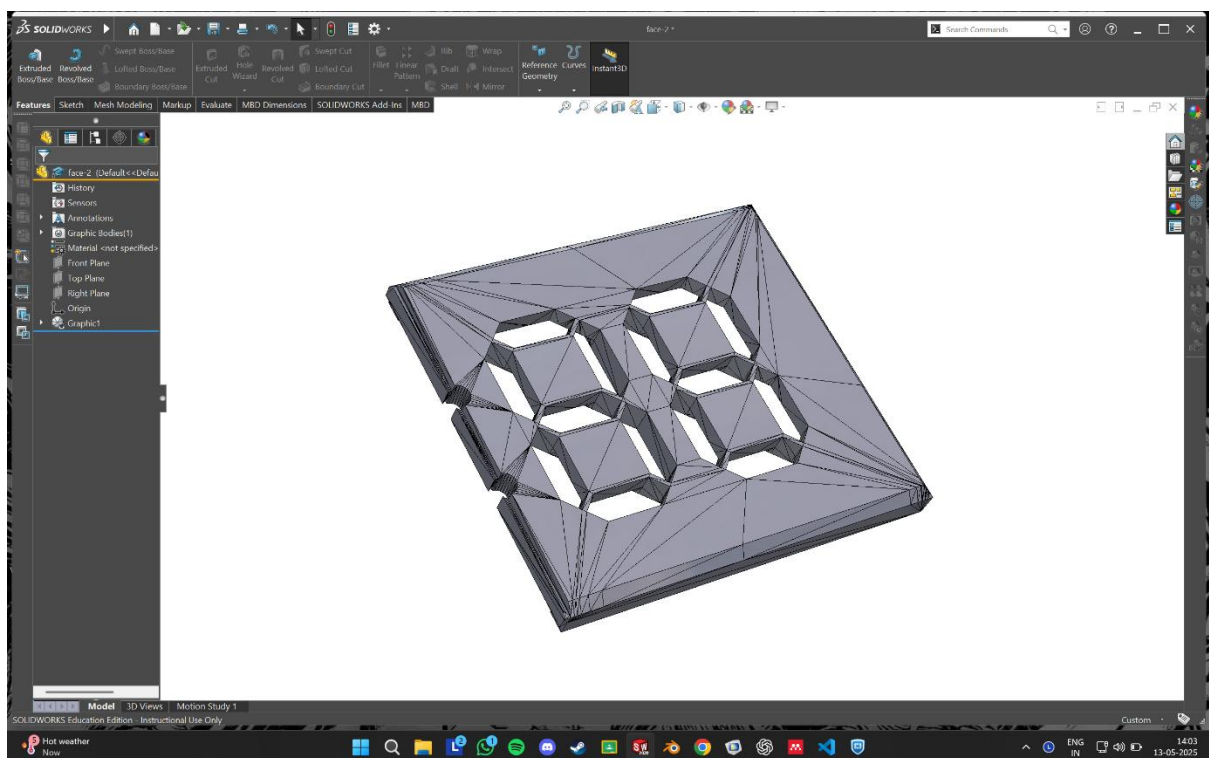
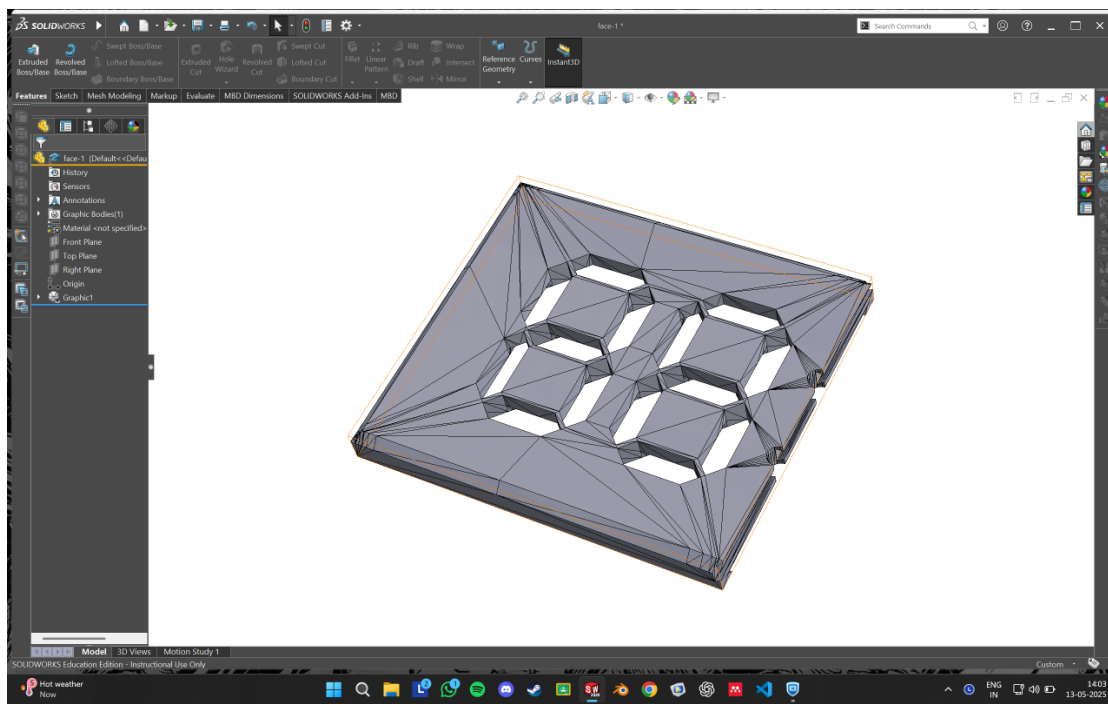


Segments:

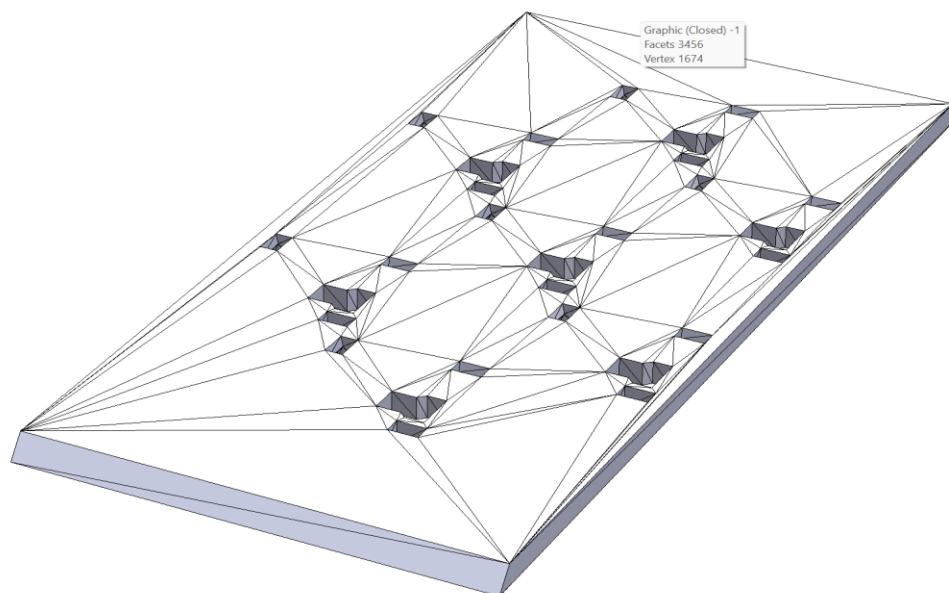
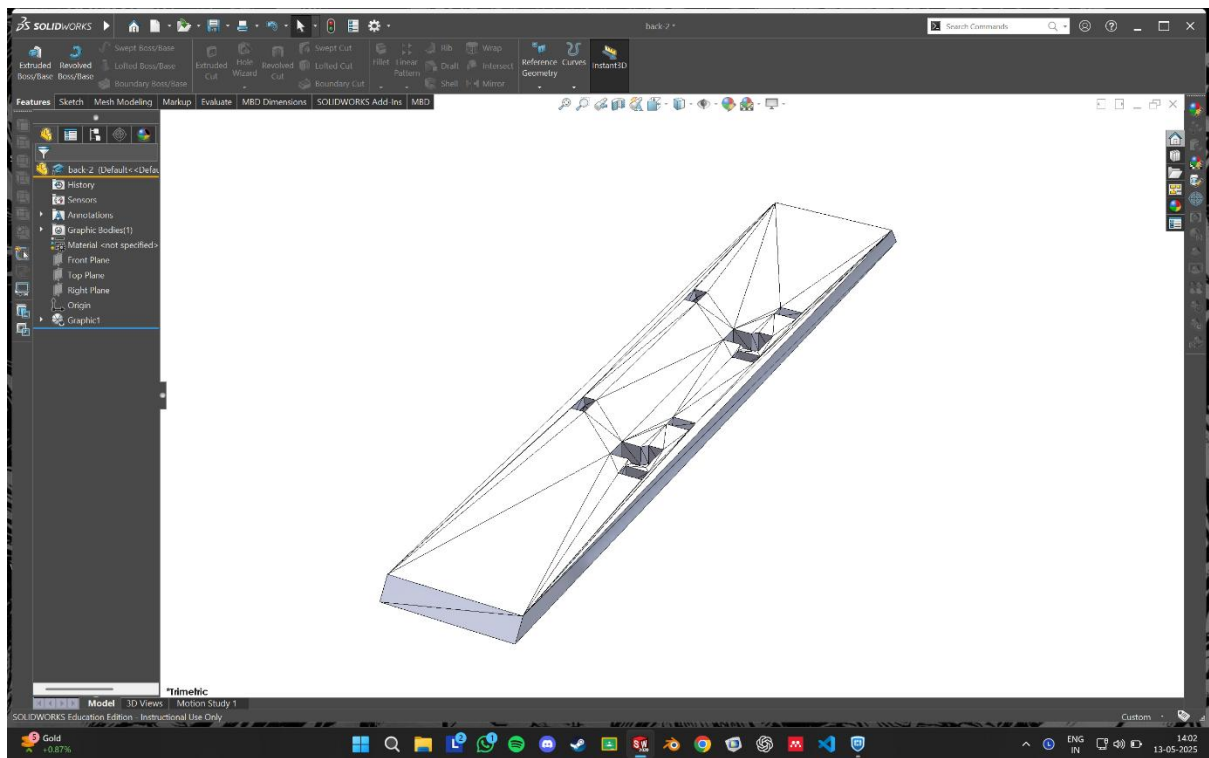




Face:



Back:



Step 2: Mount servos to Brackets. Attach all the servos to it's bracket from underneath using included screws.

Step 3: Install servo brackets to panels All the 7 segments require servo assemblies. We secure them in place with the help of Snap-fit brackets. In case of some error where you need to remove the bracket, you may just squeeze the Snap-fit tabs with pliers and that will release brackets.

Step 4: Add Panel to Main Body

Slide the segment panels into the back of the body of the clock. There are four digit panels and one panel for the colon. The body includes alignment flags, that will hold the pieces in alignment for ease of assembly.

Step 5: Connect Servos to Microcontroller

Attach servos to microcontroller. Each segment of a digit is ordered from A to G. We will map these as 1 to 7 for connecting to our microcontroller. Simply attach the servos in order, starting from the following pins.

1. The first digit will be at pin 2.
2. The second digit starts at pin 9.
3. The third digit starts at pin 22.
4. The fourth digit starts at pin 29.

You can refer to the source-code section for the final pin-out.

Step 6: Insert Segments Into the Display

These hobby servos actually only have 180 degrees of motion. In order to calibrate and align the mechanisms, prep all the servos by moving them to the end of their motion (180 degree).

Slide the segment from the front of the display into each individual socket, you will feel the servos accept the segment there should be some light resistance and smooth actuation of the part as inserting.

Step 7: Programming

Our code is run in a loop, it checks the current time and moves the destination of each segment to the desired location. Using the loop as a ticker it makes small increments on the angle of the servo creating a smooth push and pull motion on the segment.

Since the servos used in this project are cheap and not the most accurate we use the segment intervals map, to store the internal offset of each servo. This is how we ensure that when we push or pull the servos they are all aligned. This is critical to the aesthetic of the display.

Step 8: Calibration & Setup

The previous step will actually require additional calibration because the servos that I have will have completely different parameters as yours. The following code is what we used to calibrate and find the intervals needed for each servo.

```

1  ∨  /*
2      Controlling a servo position using a potentiometer (variable resistor)
3      by Michal Rinott <http://www.arduino.cc/en/Tutorial/Knob>
4      Modified on 8 Nov 2013 by Scott Fitzgerald
5      http://www.arduino.cc/en/Tutorial/Knob
6      */
7
8      #include <Servo.h>                // Include the Servo library to control servo motors
9
10     Servo myservo;                    // Create a servo object to control a single servo (used in manual mode)
11
12     int potpin = 0;                   // Analog pin (A0) connected to the potentiometer
13     int val;                           // Variable to store the analog value read from potentiometer
14
15     // Predefined angles for each segment of each digit on the 7-segment display
16     // Format: SEGMENT_INTERVALS[digit][segment][0=off, 1=on]
17  ∨  const int SEGMENT_INTERVALS[4][7][2] = {
18  ∨      {
19          {146, 57}, {147, 62}, {138, 51}, {160, 88}, {135, 64}, {149, 70}, {139, 58}
20      },
21  ∨      {
22          {131, 45}, {150, 53}, {138, 52}, {146, 61}, {151, 70}, {140, 57}, {137, 48}
23      },
24  ∨      {
25          {157, 73}, {156, 70}, {135, 50}, {137, 52}, {133, 50}, {133, 50}, {168, 73}
26      },
27  ∨      {
28          {131, 52}, {147, 61}, {131, 51}, {128, 43}, {125, 41}, {104, 24}, {137, 60}
29      }
30  };
31
32     // Predefined angles for colon LEDs (two dots between HH:MM)
33  ∨  const int COLON_INTERVAL[2][2] = {
34      {127, 45}, {156, 81}
35  };
36

```

```

35 //
36
37 // Starting pin numbers for each 7-segment digit's servo connections
38 const int DIGIT_STARTING_SEGMENT_INDEX[4] = {2, 9, 22, 29}; // Each digit uses 7 pins
39
40 const int COLON_STARTING_INDEX = 16; // Starting pin index for colon servos
41
42 const int SEGMENTS_PER_DIGIT = 7; // Each digit has 7 segments
43 const int DIGITS = 4; // Total digits in display
44 const int START_POS = 1; // Index for the "ON" angle in the data arrays
45 const int COLON = 2; // Number of colon servos (dots)
46
47 // Variable to hold the current servo pin number being manually controlled
48 int segment = 12;
49
50 void setup() {
51     Servo servos[DIGITS][SEGMENTS_PER_DIGIT]; // Array to hold servo objects for each digit's segments
52     Servo colonServos[COLON]; // Array to hold servo objects for colon dots
53
54     // === Initialize all 7-segment digits ===
55     for (int i = 0; i < DIGITS; i++) {
56         for (int j = 0; j < SEGMENTS_PER_DIGIT; j++) {
57             int offset = DIGIT_STARTING_SEGMENT_INDEX[i]; // Get the starting pin index for digit `i`
58             servos[i][j].attach(j + offset); // Attach servo to correct pin
59             servos[i][j].write(SEGMENT_INTERVALS[i][j][START_POS]); // Move segment to visible position
60         }
61
62         delay(1000); // Give time for the segments to move
63
64         // Detach servos to reduce power consumption or avoid jitter
65         for (int j = 0; j < SEGMENTS_PER_DIGIT; j++) {
66             servos[i][j].detach();
67         }
68     }
69
70     // === Initialize colon dots ===
71     for (int i = 0; i < COLON; i++) {
72         colonServos[i].attach(i + COLON_STARTING_INDEX); // Attach each colon servo to its pin
73         colonServos[i].write(COLON_INTERVAL[i][START_POS]); // Move dot to visible position
74     }
75
76     delay(1000); // Allow colon dots to settle
77
78     // Detach colon servos
79     for (int i = 0; i < COLON; i++) {
80         colonServos[i].detach();
81     }
82
83     // Attach the test servo used for manual control (controlled via serial and potentiometer)
84     myservo.attach(segment);
85
86     Serial.begin(9600); // Start serial communication for reading input
87 }
88
89 // Function to parse incoming serial string into a valid servo pin (0-180)
90 // If invalid, defaults to 0
91 int inputParse(String input) {
92     int angle = input.toInt(); // Convert string to integer
93     if (angle > 180 || angle < 0) { // Clamp value if out of range
94         angle = 0;
95     }
96     return angle;
97 }
98

```

```

70 // === Initialize colon dots ===
71 for (int i = 0; i < COLON; i++) {
72     colonServos[i].attach(i + COLON_STARTING_INDEX); // Attach each colon servo to its pin
73     colonServos[i].write(COLON_INTERVAL[i][START_POS]); // Move dot to visible position
74 }
75
76 delay(1000); // Allow colon dots to settle
77
78 // Detach colon servos
79 for (int i = 0; i < COLON; i++) {
80     colonServos[i].detach();
81 }
82
83 // Attach the test servo used for manual control (controlled via serial and potentiometer)
84 myservo.attach(segment);
85
86 Serial.begin(9600); // Start serial communication for reading input
87 }
88
89 // Function to parse incoming serial string into a valid servo pin (0-180)
90 // If invalid, defaults to 0
91 int inputParse(String input) {
92     int angle = input.toInt(); // Convert string to integer
93     if (angle > 180 || angle < 0) { // Clamp value if out of range
94         angle = 0;
95     }
96     return angle;
97 }
98

```

```

99  void loop() {
100    // === Check for serial input to change the servo pin ===
101    if (Serial.available()) {
102        segment = inputParse(Serial.readString()); // Parse serial string into a valid pin number
103
104        Serial.print("Attaching: ");
105        Serial.println(segment);
106
107        myservo.detach(); // Detach the current servo (if any)
108        myservo.attach(segment); // Attach to the new servo pin
109    }
110
111    // === Control the current servo using potentiometer ===
112    if (myservo.attached()) {
113        val = analogRead(potpin); // Read value from potentiometer (0-1023)
114        val = map(val, 0, 1023, 0, 180); // Map it to angle range for servo
115        myservo.write(val); // Set the servo to new angle
116
117        // Print debug info to Serial Monitor
118        Serial.print("Seg: ");
119        Serial.print(segment);
120        Serial.print(" Pot: ");
121        Serial.println(val);
122
123        delay(15); // Wait for the servo to reach new position
124    }
125 }
126

```

In addition, we also need to set the time of our RTC. Here is the following code used to do that.

```

1  // DS3231_Serial_Easy
2  // Copyright (C)2015 Rinky-Dink Electronics, Henning Karlsen. All rights reserved
3  // Web: http://www.RinkyDinkElectronics.com
4
5  // A quick demo of how to use the DS3231-library to
6  // send time and date information over Serial monitor
7
8  // =====
9  // === I2C Wiring Reference ===
10 // =====
11 //
12 // Arduino Uno/2009:
13 //   DS3231 SDA -> A4
14 //   DS3231 SCL -> A5
15 //
16 // Arduino Leonardo:
17 //   DS3231 SDA -> D2
18 //   DS3231 SCL -> D3

```

```

20 // Arduino Mega:
21 //   DS3231 SDA -> D20 (SDA)
22 //   DS3231 SCL -> D21 (SCL)
23 //
24 // Arduino Due:
25 //   DS3231 SDA -> D20 (SDA) or SDA1 (D70)
26 //   DS3231 SCL -> D21 (SCL) or SCL1 (D71)
27 //
28 // Note:
29 // - When using hardware I2C pins, internal pull-up resistors are used.
30 // - If using other pins (non-standard), library switches to software I2C.
31 //   In that case, you must add external pull-ups on SDA and SCL lines.
32
33 #include <DS3231.h> // Include the DS3231 RTC library
34
35 // Create an instance of the DS3231 object using hardware I2C pins
36 // SDA and SCL are default I2C pins defined by the board
37 DS3231 rtc(SDA, SCL);
38
39 void setup()
40 {
41     // Start serial communication at 115200 baud
42     Serial.begin(115200);
43
44     // For Arduino Leonardo only: wait for serial port to connect
45     // (uncomment below if using Leonardo)
46     // while (!Serial) {}
47
48     // Initialize the DS3231 RTC module
49     rtc.begin();
50
51     // =====
52     // === OPTIONAL: Set RTC Time ==
53     // =====
54
55     // These lines are for setting the RTC's date and time.
56     // Uncomment and modify only ONCE when setting the time.
57     // Comment them back out after the RTC is correctly set.

```

```

58
59     // rtc.setDOW(FRIDAY);    // Set the day of the week (e.g., SUNDAY, MONDAY, ...)
60     rtc.setTime(23, 05, 0);   // Set time: hour (0-23), minute, second
61     rtc.setDate(7, 19, 2021); // Set date: month (1-12), day, year
62 }
63
64 void loop()
65 {
66     // Print the current time in "HH:MM:SS" format
67     Serial.println(rtc.getTimeStr());
68
69     // Wait for 1 second before printing again
70     delay(1000);
71 }
72

```

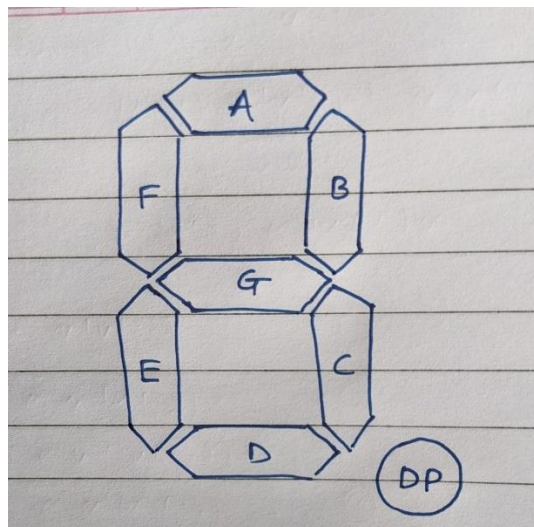
Wiring part:

For rtc module

SDA (Serial Data Line) → Arduino Mega Digital Pin 20 (SDA)

SCL (Serial Clock Line) → Arduino Mega Digital Pin 21 (SCL)

For servos



Attach servos to microcontroller. Each segment of a digit is ordered from A to G. We will map these as 1 to 7 for connecting to our microcontroller. Simply attach the servos in order, starting from the following pins.

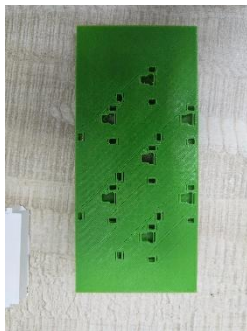
- The first digit will be at pin 2.
- The second digit starts at pin 9.
- The third digit starts at pin 22.
- The fourth digit starts at pin 29.

The only differences that were done in the above process:

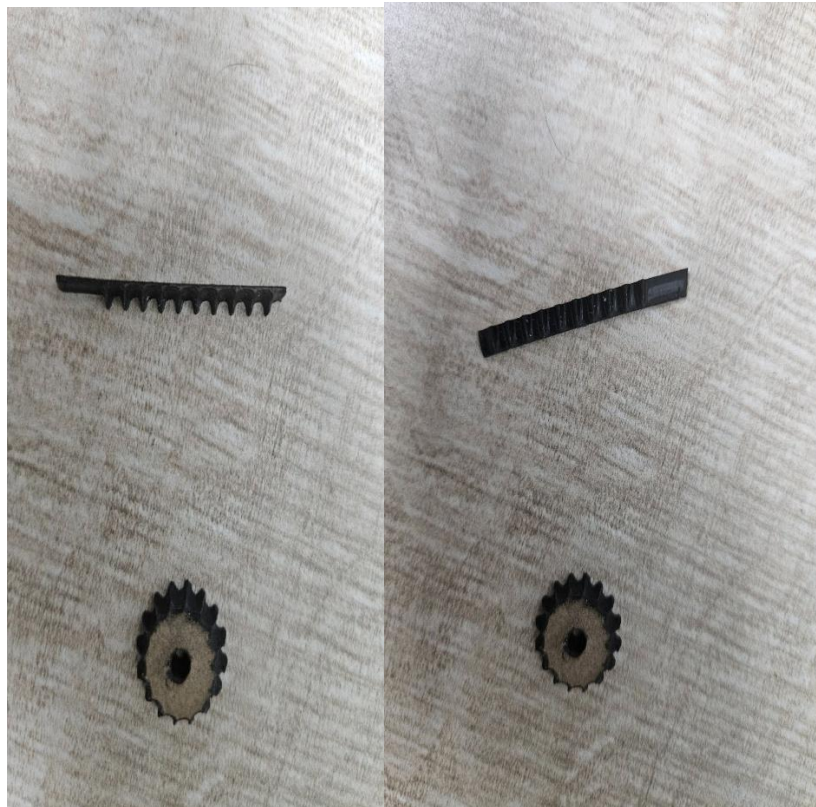
Rack and Pinions are cut using a CO2 laser cutter

Problems in the way:

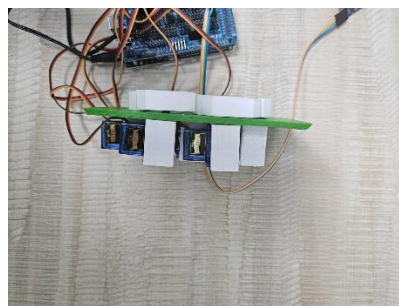
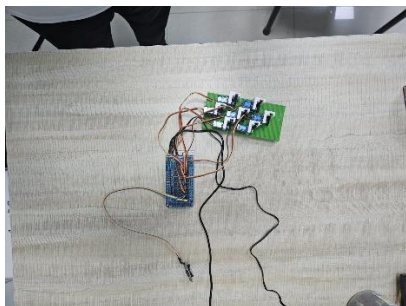
1. The 3d printer had very poor print quality, so removing supports was a nightmare. And without the supports, it wasn't printing too so we had no option but to go with the supports. Also, the 3d printers aren't assembled properly. All of the filament dispensers were on the wrong side. We fixed the one we used. Possibly, there are more issues that we don't know about in the assembly.



2. Parts weren't fitting into each other.
3. The laser cutter made a bevel-like structure on the rack and pinion, and also, they were smaller than the required size, so it wasn't meshing properly.



4. Need to recalibrate the Real Time clock module if the clock's battery is discharged.



5. Accidentally glued a servo's shaft to itself.

Solutions:

1. Using a more error-resistant design that also works with some wrong dimensions
2. Checking and verifying the accuracy and precision of the 3d printer and laser printer beforehand
3. Using a CO2 laser cutter for the face of the clock to avoid dimensional errors and also save material.
4. Using CNC for certain parts that require high dimensional accuracy.

5. Adding physical buttons to set the time to avoid recalibration using computer code.
6. Design the clock such that minimal glue is used and that too in an area where spilling some of the glue wouldn't cause any issues.

CODE:-

```
1  #include <Servo.h>      // Library for controlling servo motors
2  #include <Wire.h>       // Library for I2C communication
3  #include <DS3231.h>     // Library to interface with the DS3231 Real-Time Clock (RTC)
4
5  DS3231 rtc;            // Create an RTC object
6
7  // === Digit to Segment Mapping ===
8  // Each row corresponds to a digit (0-9).
9  // Each column (7 values) corresponds to segments A-G (1 = visible, 0 = hidden).
10 const int DIGIT_TO_SEGMENT_MAPPING[10][7] = {
11     { 1, 1, 1, 1, 1, 1, 0 }, // 0
12     { 0, 1, 1, 0, 0, 0, 0 }, // 1
13     { 1, 1, 0, 1, 1, 0, 1 }, // 2
14     { 1, 1, 1, 1, 0, 0, 1 }, // 3
15     { 0, 1, 1, 0, 0, 1, 1 }, // 4
16     { 1, 0, 1, 1, 0, 1, 1 }, // 5
17     { 1, 0, 1, 1, 1, 1, 1 }, // 6
18     { 1, 1, 1, 0, 0, 0, 0 }, // 7
19     { 1, 1, 1, 1, 1, 1, 1 }, // 8
20     { 1, 1, 1, 1, 0, 1, 1 }  // 9
21 };
22
23 // === Servo Position Intervals ===
24 // Each digit (4 total), each segment (7 per digit), has two servo positions:
25 // [0] = hidden position, [1] = visible position
26 const int SEGMENT_INTERVALS[4][7][2] = {
27     { {141, 54}, {155, 69}, {150, 73}, {151, 70}, {159, 75}, {159, 75}, {125, 40} },
28     { {164, 76}, {155, 76}, {138, 61}, {180, 87}, {151, 63}, {145, 57}, {165, 78} },
29     { {157, 73}, {156, 70}, {165, 85}, {137, 52}, {133, 50}, {133, 50}, {168, 73} },
30     { {131, 52}, {147, 61}, {131, 51}, {158, 69}, {155, 73}, {116, 28}, {137, 60} }
31 };
32
33 // === Colon Servo Position Intervals ===
34 // [0] = hidden, [1] = visible for both colon dots
35 const int COLON_INTERVAL[2][2] = { {141, 62}, {137, 30} };
```

```

37 // === Configuration Parameters ===
38 const int DIGIT_START_INDEX[4] = {2, 9, 22, 29}; // Starting servo pin numbers for each digit
39 const int COLON_INDEX = 16; // Starting pin for the colon (middle dots)
40 const int NUM_DIGITS = 4; // Total digits (HHMM)
41 const int NUM_SEGMENTS = 7; // Segments per digit
42 const int STEP_DELAY = 20; // Delay between segment updates (ms)
43 const int COUNT_MS = 2000; // Interval for refreshing the display (ms)
44
45 int servoDest[NUM_DIGITS][NUM_SEGMENTS]; // Target positions for all segments
46 int count = 1200; // Internal counter (not used significantly here)
47 unsigned long lastUpdateTime = 0; // Tracks last time display was updated
48
49 Servo digits[NUM_DIGITS][NUM_SEGMENTS]; // Servo objects for digit segments
50 Servo colon[2]; // Servo objects for the colon (2 dots)
51
52 // === Attach all digit servos and set them to their default positions ===
53 void attachServos() {
54     for (int d = 0; d < NUM_DIGITS; d++) {
55         for (int s = 0; s < NUM_SEGMENTS; s++) {
56             digits[d][s].attach(DIGIT_START_INDEX[d] + s); // Attach servo to pin
57             digits[d][s].write(SEGMENT_INTERVALS[d][s][0]); // Move to hidden position
58         }
59     }
60     delay(500); // Wait for servos to reach position
61     detachServos(); // Detach to reduce power and jitter
62 }
63
64 // === Attach colon servos and move to default position ===
65 void attachColon() {
66     for (int i = 0; i < 2; i++) {
67         colon[i].attach(COLON_INDEX + i);
68         colon[i].write(COLON_INTERVAL[i][0]); // Move to hidden position
69     }
70     delay(500);
71     detachColon();
72 }
73

```

```

73
74 // === Detach digit servos to conserve power ===
75 void detachServos() {
76     for (int d = 0; d < NUM_DIGITS; d++) {
77         for (int s = 0; s < NUM_SEGMENTS; s++) {
78             digits[d][s].detach();
79         }
80     }
81 }
82
83 // === Detach colon servos to conserve power ===
84 void detachColon() {
85     for (int i = 0; i < 2; i++) {
86         colon[i].detach();
87     }
88 }
89
90 // === Arduino Setup Function ===
91 void setup() {
92     Wire.begin(); // Start I2C
93     rtc.setClockMode(false); // Set RTC to 24-hour mode
94     attachServos(); // Attach and initialize digit servos
95     attachColon(); // Attach and initialize colon servos
96 }
97
98 // === Update the target positions of each segment based on the current time ===
99 void updateDisplay() {
100     bool h12, PM_time;
101     byte hour = rtc.getHour(h12, PM_time); // Get current hour
102     byte minute = rtc.getMinute(); // Get current minute
103     String timeStr = String(hour) + String(minute); // Concatenate as string (e.g., "937")
104
105     Serial.print("Time: "); // Debug print to Serial Monitor
106     Serial.println(timeStr);
107

```

```

107
108 // Loop through each digit and assign target positions for each segment
109 ∨ for (int d = 0; d < NUM_DIGITS; d++) {
110     int numberToShow = timeStr.charAt(timeStr.length() - 1 - d) - '0'; // Get digit value
111
112 ∨     for (int s = 0; s < NUM_SEGMENTS; s++) {
113         int posIndex = DIGIT_TO_SEGMENT_MAPPING[numberToShow][s]; // 0 or 1
114         servoDest[d][s] = SEGMENT_INTERVALS[d][s][posIndex]; // Assign corresponding angle
115     }
116 }
117 }
118
119 // === Move segments smoothly toward their target positions ===
120 ∨ void moveSegments() {
121 ∨     for (int d = 0; d < NUM_DIGITS; d++) {
122 ∨         for (int s = 0; s < NUM_SEGMENTS; s++) {
123             int currentPos = digits[d][s].read(); // Current angle
124             int targetPos = servoDest[d][s]; // Target angle
125
126 ∨             if (currentPos != targetPos) {
127                 digits[d][s].attach(DIGIT_START_INDEX[d] + s); // Attach if movement needed
128                 currentPos += (currentPos < targetPos) ? 1 : -1; // Increment or decrement
129                 digits[d][s].write(currentPos); // Write new position
130             }
131         }
132     }
133 }
134
135 // === Main Loop ===
136 ∨ void loop() {
137     // Check if enough time has passed to update the display
138 ∨     if (millis() - lastUpdateTime >= STEP_DELAY) {
139         updateDisplay(); // Fetch time and update servo targets
140         moveSegments(); // Move servos smoothly toward target
141         lastUpdateTime = millis();
142     }
143
144     // Count refresh interval (not essential here)
145 ∨     if (millis() - lastUpdateTime >= COUNT_MS) {
146         count++;
147     }
148
149     detachServos(); // Detach servos regularly to reduce power draw and noise
150 }
151
152 // === Wiring Note ===
153 // SDA (Data Line) -> Arduino Mega Pin 20
154 // SCL (Clock Line) -> Arduino Mega Pin 21
155

```

The above Arduino code powers a mechanical digital clock using servo motors and a DS3231 real-time clock (RTC) module. It reads the current time, translates each digit into a 7-segment display pattern, and moves servos to show or hide each segment accordingly. The servos are updated smoothly to reflect the current time, and are detached when idle to save power and reduce noise. This creates a functional clock with physical, moving parts instead of LED displays.

Overall it was wholesome project which made us understand how in theory everything seems to work out but in real world it's much more complex and needs a lot of thinking and presence of mind in order for it to function.